



MINISTRY OF EDUCATION, CULTURE AND RESEARCH

OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Report

on laboratory work no. 2

of Operating Systems

Topic: Meet the OS You Will Build

Done by: st. gr. FAF-243, Augustin Carazanu

Checked by: assist. univ. Patricia Reîțman

FCIM, UTM

Chișinău – 2026

Purpose of the work

The purpose of this laboratory work is to build the xv6 teaching operating system (a small re-implementation of Unix Version 6 for RISC-V) from source, boot it in the QEMU emulator, use it as a small Unix system, read its source code to understand the separation between the kernel and user programs, and write a first user program that makes a system call into the kernel.

Before you start

The course VM image was not available, so the RISC-V cross-compiler and QEMU were installed on my own Ubuntu 24.04.3 ARM64 VM with apt. The xv6 source code was then cloned from the official MIT repository.

```
sudo apt install -y git build-essential gdb-multiarch qemu-system-misc gcc-
riscv64-linux-gnu binutils-riscv64-linux-gnu
git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv && ls
kernel LICENSE Makefile mkfs README test-xv6.py user
```

```
Setting up libgcc-13-dev-riscv64-cross (13.3.0-6ubuntu2-24.04.1cross1) ...
Setting up gcc-13-riscv64-linux-gnu (13.3.0-6ubuntu2-24.04.1cross1) ...
Setting up gcc-riscv64-linux-gnu (4:13.2.0-7ubuntu1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for gnome-menus (3.36.0-1ubuntu3) ...
Processing triggers for libc-bin (2.39-0ubuntu8.5) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for desktop-file-utils (0.27-2build1) ...
parallels@ubuntu-gnu-linux-24-04-3:~$ git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
Cloning into '/home/parallels/xv6-riscv'...
remote: Enumerating objects: 8334, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 8334 (delta 19), reused 11 (delta 11), pack-reused 8301 (from 2)
Receiving objects: 100% (8334/8334), 17.67 MiB | 3.25 MiB/s, done.
Resolving deltas: 100% (4583/4583), done.
kernel LICENSE Makefile mkfs README test-xv6.py user
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$
```

Figure 1.1 – Installing the RISC-V toolchain and cloning the xv6 source code

Part 1. Build it and boot it

make qemu compiled the kernel and every user program with riscv64-linux-gnu-gcc. It then built the file-system image fs.img with mkfs, and started QEMU emulating a RISC-V machine with 128 MB of RAM and 3 CPU cores (-smp 3). The kernel printed xv6 kernel is booting, started the other two cores (hart 1 starting, hart 2 starting), and the first process init launched the shell.

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 1.2 – Building xv6 with make qemu and booting the kernel

Part 2. Use xv6 as the Unix it is

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2441
cat        2 3 35376
echo      2 4 34312
forktest  2 5 17280
grep       2 6 42736
init       2 7 34768
kill       2 8 34240
ln         2 9 34032
ls         2 10 41520
mkdir     2 11 34280
rm         2 12 34264
sh         2 13 56136
stressfs  2 14 35160
usertests  2 15 200056
grind      2 16 50504
wc         2 17 36336
zombie    2 18 33648
logstress  2 19 36184
forphan   2 20 35048
dorphan   2 21 34488
sync      2 22 33712
console   3 23 0
```

Figure 2.1 – Listing the files and programs inside xv6 with ls

```
$
$ echo hello xv6
wc READMEhello xv6
$
48 336 2441 README
$
```

Figure 2.2 – Running `echo` and `wc` inside `xv6`

```
$
$ ls | grep c
cat          2 3 35376
echo         2 4 34312
wc           2 17 36336
sync         2 22 33712
console      3 23 0
$ QEMU: Terminated
```

Figure 2.3 – Using a pipe inside `xv6` (`ls | grep c`)

Interesting output:

```
$ wc README
48 336 2441 README
```

`xv6`'s `ls` prints four columns: the name, the type (1 = directory, 2 = file, 3 = device), the inode number and the size in bytes. `console` is a device (type 3) with size 0. This is the same "everything is a file" idea as in Lab 1.

(1) List three programs `xv6` ships with.

`cat`, `echo` and `grep`. Others include `ls`, `wc`, `mkdir`, `rm`, `kill`, `sh` and `usertests`.

(2) Which two OS features must exist for a pipe to work?

- **Process creation (fork and exec).** The shell must start `ls` and `grep` as two separate processes that run at the same time.
- **Inter-process communication (the pipe system call).** The kernel creates a buffer with a write end and a read end, which are file descriptors. `ls` writes its output into one end, and `grep` reads it as input from the other. The shell uses `close` and `dup` to connect them to standard output and standard input.

(3) How does the `xv6` shell compare to the Linux shell from Lab 1?

It supports the same basic Unix syntax (commands with arguments, pipes `|`, redirection `<` `>` and `&`). However, it is much simpler than Linux `bash`: it has no command history, tab completion, variables or job control, and its prompt is only `$`.

Part 3. Read the source

```
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
    static char buf[DIRSIZ + 1];
    char *p;

    // Find first character after last slash.
    for (p = path + strlen(path); p >= path && *p != '/'; p--)
        ;
    p++;

    // Return blank-padded name.
    if (strlen(p) >= DIRSIZ)
        return p;
    memmove(buf, p, strlen(p));
    memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
    buf[sizeof(buf) - 1] = '\0';
    return buf;
}

void
ls(char *path)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    if ((fd = open(path, O_RDONLY)) < 0) {
        fprintf(2, "ls: cannot open %s\n", path);
        return;
    }

    if (fstat(fd, &st) < 0) {
```

Figure 3.1 – Source code of the user program ls (user/ls.c)

```
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
    int n;

    while ((n = read(fd, buf, sizeof(buf))) > 0) {
        if (write(1, buf, n) != n) {
            fprintf(2, "cat: write error\n");
            exit(1);
        }
    }
    if (n < 0) {
        fprintf(2, "cat: read error\n");
        exit(1);
    }
}

int
main(int argc, char *argv[])
{
    int fd, i;

    if (argc <= 1) {
        cat(0);
    }
}
```

Figure 3.2 – Source code of cat, using the read() and write() system calls

```

parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ cat user/user.h
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);

```

Figure 3.3 – System calls available to user programs (user/user.h)

```

parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$

```

Figure 3.4 – Locating sys_read and sys_write in kernel/sysfile.c

Interesting output:

```

$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)

```

(1) Which system calls does user/cat.c use, and what does each one ask the kernel to do?

- `read(fd, buf, sizeof(buf))` asks the kernel to copy up to 512 bytes from the open file `fd` into the program's buffer. It returns the number of bytes read, 0 at the end of the file, or a negative value on error.
- `write(1, buf, n)` asks the kernel to write `n` bytes from the buffer to file descriptor 1 (standard output, the console).

- `open(path, O_RDONLY)` asks the kernel to open a file by name and return a new file descriptor. `close(fd)` releases that descriptor. Both are used in `main` for each file given as an argument.
- `exit(status)` asks the kernel to end the process and free its resources.

`fprintf(2, ...)` is not a system call. It is a library function (`user/printf.c`) that formats the text and then calls `write` on file descriptor 2 (standard error).

(2) In which kernel file, and on which line, is `sys_read` implemented?

In `kernel/sysfile.c`, on line 69 (`sys_read(void)`). `sys_write` is on line 83 of the same file.

(3) What is the difference between the code in `kernel/` and the code in `user/`?

The code in `kernel/` is the operating system itself. It runs in privileged supervisor mode with full access to the hardware and memory. The code in `user/` runs in unprivileged user mode, and it can only get services such as files, memory and devices by asking the kernel through system calls.

`user/user.h` is the full list of these requests: `fork`, `exit`, `wait`, `pipe`, `read`, `write`, `open`, `close`, `exec`, `pause` and the others.

Part 4. Write your first xv6 program

The program `user/sleep.c` was created, and `$U/_sleep\` was added to the `UPROGS` list in the `Makefile`, right after `$U/_echo\`.

```

parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ cat > user/sleep.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    sleep(atoi(argv[1])); // system call into the kernel
    exit(0);
}
EOF
cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    sleep(atoi(argv[1])); // system call into the kernel
    exit(0);
}
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$

```

Figure 4.1 – Creating the new user program user/sleep.c

```

parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ sed -i '/\$_echo\\a\\tSU/_sleep\\' Makefile
grep -n "_echo\\_sleep" Makefile
134:  $U/_echo\
135:  $U/_sleep\
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$

```

Figure 4.2 – Adding `_sleep` to the `UPROGS` list in the `Makefile`

Problem encountered. The first build failed with error: implicit declaration of function 'sleep'. The current version of xv6-riscv renamed the sleep system call to pause, which can be seen in user/user.h (int pause(int);). The lab sheet was written for the older name. Because xv6 compiles with -Werror, the undeclared function stopped the build.

```

    exit(0);
}
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ sed -i '/\$_echo\\a\\tSU/_sleep\\' Makefile
grep -n "_echo\\_sleep" Makefile
134:  $U/_echo\
135:  $U/_sleep\
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/parallels/xv6-riscv- -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno-builtin-protector -fno-pie -no-pie -c -o user/sleep.o user/sleep.c
user/sleep.c: In function 'main':
user/sleep.c:11:3: error: implicit declaration of function 'sleep' [-Werror=implicit-function-declaration]
   11 |     sleep(atoi(argv[1])); // system call into the kernel
      |     ^~~~~
At top level:
cc1: note: unrecognized command-line option '-Wno-unknown-attributes' may have been intended to silence earlier diagnostics
cc1: all warnings being treated as errors
make: *** [: user/sleep.o] Error 1
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$

```

Figure 4.3 – Build error: the `sleep()` system call is not declared

Fix. The system call in sleep.c was changed from sleep(. . .) to pause(. . .). The user program is still called sleep. After this change, the kernel and _sleep compiled, _sleep was included in fs.img, and xv6 booted.

```

make: *** [-builtin: user/sleep.o] Error 1
parallels@ubuntu-gnu-linux-24-04-3:~/xv6-riscv$ sed -i 's/sleep(atoi/pause(atoi/' user/sleep.c
cat user/sleep.c
make qemu
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1])); // system call into the kernel
    exit(0);
}
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/parallels/xv6-riscv-. -march=rv64gc -std=gnu99 -MD
-mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp
-fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-prin
tf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_sleep user/sleep.o user/lib.o user/usys.o user/printf.o user/unalloc.o
riscv64-linux-gnu-objdump -S user/_sleep > user/sleep.asm
riscv64-linux-gnu-objdump -t user/_sleep | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/sleep.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_sleep user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_use
r/_tests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 988 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -n 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -
device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$

```

Figure 4.4 – Replacing sleep() with pause(), rebuilding and booting xv6

Final user/sleep.c:

```

#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1])); // system call into the kernel
    exit(0);
}

```

Test. `sleep 10` paused for 10 clock ticks (about one second) and then returned to the prompt. `sleep` with no argument printed the usage message.

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
sleep
$ usage: sleep <ticks>
$

```

Figure 4.5 – Testing the sleep program inside xv6

```

$ sleep 10
$ sleep
usage: sleep <ticks>

```

Conclusions

In this lab I built xv6 from source, booted it on an emulated RISC-V machine, and used it as a small Unix. Its commands, pipes and files work like the Linux system from Lab 1.

Reading the source showed the main split in an operating system. Programs in `user/` only make requests through system calls such as `read`, `write` and `pause`, and the kernel code in `kernel/` carries them out. `sys_read`, for example, is on line 69 of `kernel/sysfile.c`.

Finally, I wrote my own user program, `sleep`, that makes a real system call into a kernel I compiled myself. Along the way I had to adapt it to the renamed `pause` system call in the current xv6.